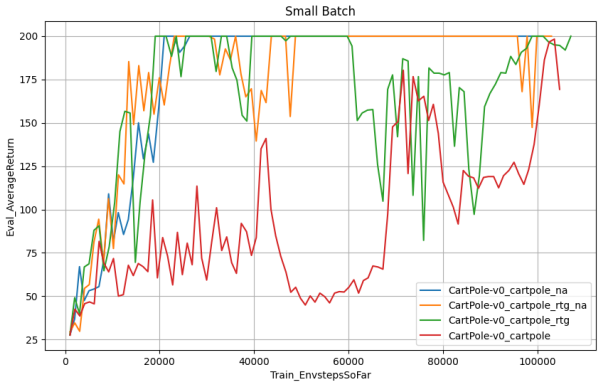
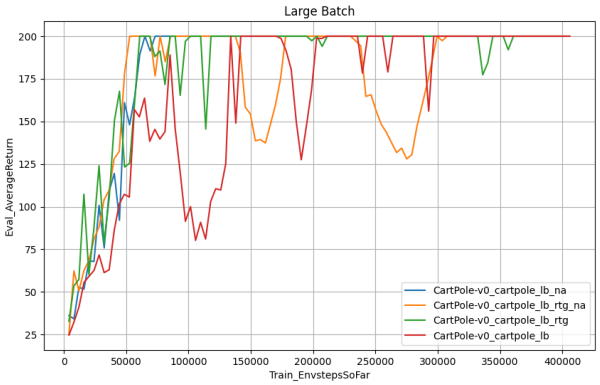
Pt 3. (Exp 1. CartPole)





In general, it would seem that the reward-to-go model performed better than a full-trajectory one. Most applications would see preference for reward-to-go over trajectory-based methods, as using reward-to-go essentially puts more focus on the "current" state and action; mathematically, subtracting the baseline helps reduce variance. As for normalizing advantages, it doesn't exactly help the model converge faster in this particular experiment, most likely due to the simplistic nature of the environment, but we can see that generally it does keep the model more stable; ie experiments run with the -na flag stay at optimal performance more consistently after learning, while without it we see dips back to non-optimal performance in the small-batch experiments. The large-batch experiments are generally more stably learning than the small-batch ones.

Commands:

uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 --exp_name cartpole

uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg --exp_name cartpole_rtg

uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -na --exp_name cartpole_na

uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg -na --exp_name cartpole_rtg_na

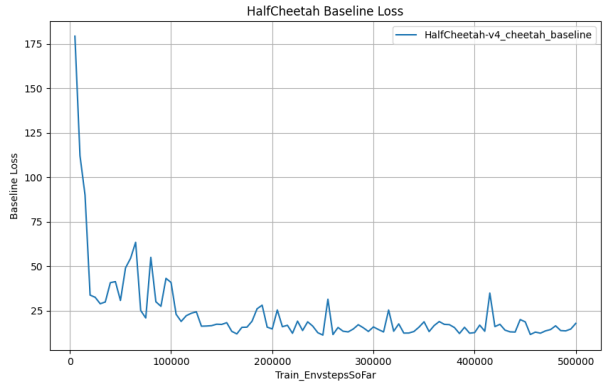
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 --exp_name cartpole_lb

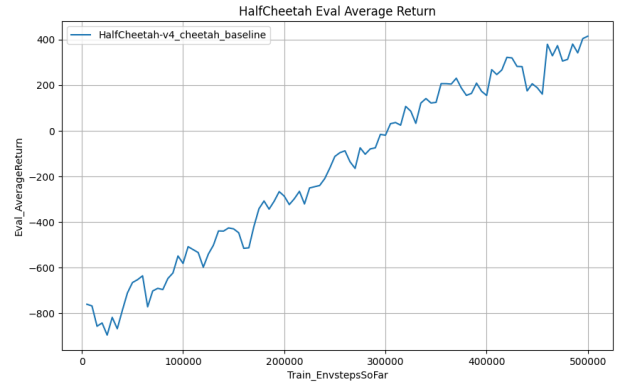
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg --exp_name cartpole_lb_rtg

uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -na --exp_name cartpole_lb_na

uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg -na --exp_name cartpole_lb_rtg_na

Pt 4. (Exp 2. HalfCheetah)





Reducing the baseline gradient steps, performance noticeably degraded—from 5 to 3 to finally 1 step, there was a clear decline in how long it took the model to hit 200 in Eval_AverageReturn. The baseline loss was also noisier; instead of a clean downwards curve, there were a few spikes in loss throughout until around 30k training steps. Adding in the -na showed the greatest improvement, hitting over 200 average returns the soonest.

Commands:

uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --exp_name cheetah

uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 5 --exp_name cheetah_baseline

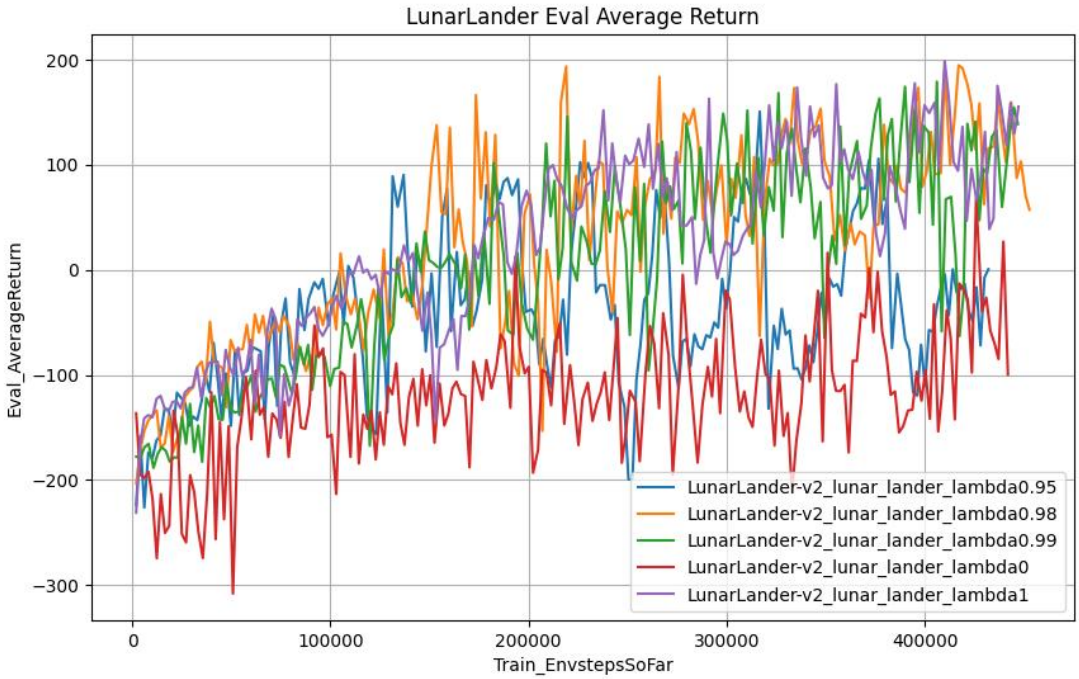
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 3 --exp_name cheetah_baseline_bgs3

uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 1 --exp_name cheetah_baseline_bgs1

uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.005 -bgs 5 --exp_name cheetah_baseline_blr005

uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg -na --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 5 --exp_name cheetah_baseline_na

Pt 5. (Exp 3. LunarLander)



As lambda increased, we see faster and more stable learning, although the highest rewards early on were achieved not by lambda=1 but 0.98. Lambda is essentially a modulator for short- vs long-term rewards, where lambda=0 means pure focus on short-term estimates and lambda=1 allows for more weight put on long-term estimates. In this environment, it makes sense for a strong emphasis on future returns to foster a stronger policy, because the LunarLander environment has a clear long-term goal that is directly influenced by what is done early on; at lambda=0 the focus purely on what immediately would happen next often doesn't align with the best overall trajectory.

Commands:

uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0 --exp_name lunar_lander_lambda0

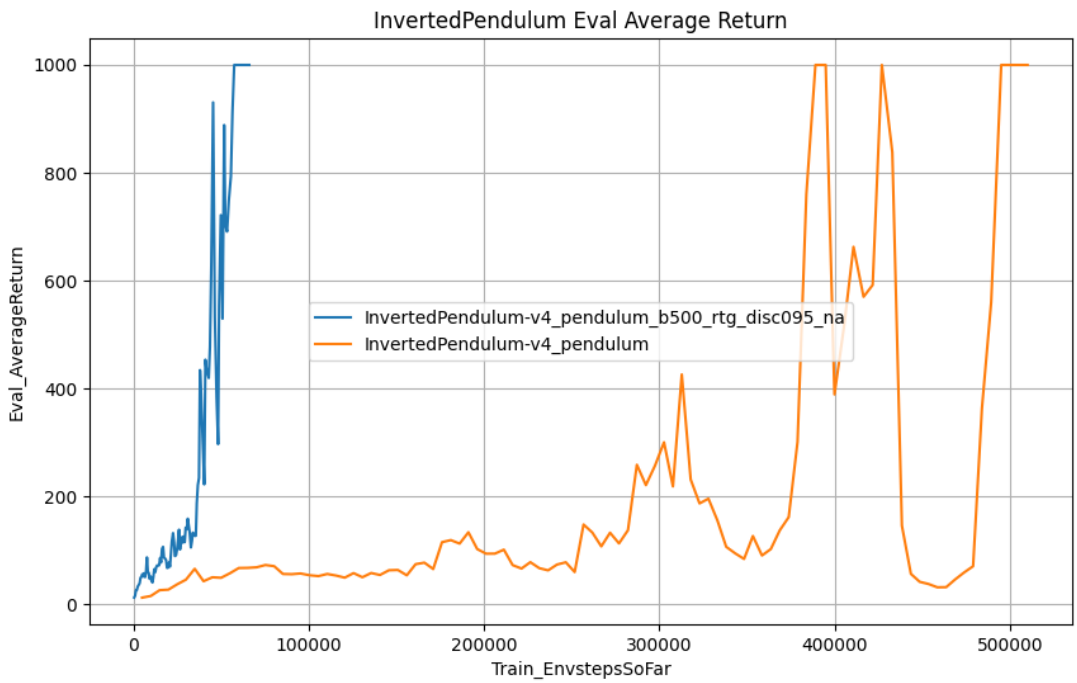
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0.95 --exp_name lunar_lander_lambda095

uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0.98 --exp_name lunar_lander_lambda098

uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0.99 --exp_name lunar_lander_lambda099

uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 1 --exp_name lunar_lander_lambda1

Pt 6. (Exp 4. InvertedPendulum)



The hyperparameter that mattered the most was batch size: in general, the model could learn after around 60 episodes, so a smaller batch size was necessary to reduce unnecessary training steps. I found a batch size of 500 to be the sweet spot—and more was redundant; any less was too noisy.

Using a discount factor of 0.95 helped it slightly prioritize keeping the pendulum stable in the immediately following timesteps, while reward-to-go and normalizing the advantage were necessary to keep the reward signal clean. However, in testing, a discount factor of 0.95 and batch size between 500 and 1000 was in general sufficient for the requirements.

Commands:

uv run src/scripts/run.py --env_name InvertedPendulum-v4 -n 100 -b 500 -eb 2000 -rtg -na --discount 0.95 --exp_name pendulum_b500_rtg_disc095_na